🏛️ Arquitetura do Compilador TinyScript

> **Disciplina:** Teoria da Computação e Compiladores — UNIBH 2026/1
> **Professora:** Charlene Cássia de Resende
> **Aluno:** Sérgio Pinton Pavanelli | **RA:** 123220202
> **Entregável 2:** Projeto da arquitetura do compilador proposto

🎯 Objetivos deste documento

- Especificar formalmente a linguagem TinyScript
- Descrever a arquitetura em camadas do compilador
- Apresentar o fluxo de dados entre as fases
- Documentar as decisões de projeto

1. 💡 Visão Geral — O que é TinyScript?

TinyScript é uma linguagem de programação imperativa minimalista criada para fins didáticos. Ela expõe de forma clara os conceitos fundamentais de compilação:

- **Análise Léxica** — reconhecimento de tokens
- **Análise Sintática** — verificação de estrutura gramatical e construção da AST
- **Geração de Código** — produção de código intermediário de três endereços (TAC)

A linguagem suporta variáveis inteiras, operações aritméticas, comparações, estruturas de controle e impressão de valores.

2. 🗂️ Especificação da Linguagem TinyScript

2.1 Tipos de Dados

Tipo	Descrição	Exemplo
`inteiro`	Número inteiro de precisão arbitrária	`42` , `0` , `-10`

> **Nota:** TinyScript suporta apenas inteiros nesta versão. Extensões para `float` e `string` são previstas em versões futuras.

2.2 Tokens (Unidades Léxicas)

Categoria	Exemplos	Descrição
`INTEGER`	`0` , `42` , `1000`	Literal inteiro
`IDENT`	`x` , `total` , `resultado_1`	Identificador
`IF`	`if`	Palavra reservada
`ELSE`	`else`	Palavra reservada
`WHILE`	`while`	Palavra reservada
`PRINT`	`print`	Palavra reservada
`PLUS` / `MINUS`	`+` , `-`	Operadores aritméticos
`STAR` / `SLASH`	`*` , `/`	Operadores aritméticos
`ASSIGN`	`=`	Atribuição
`EQ` / `NEQ`	`==` , `!=`	Comparação de igualdade
`LT` / `GT` / `LE` / `GE`	`<` , `>` , `<=` , `>=`	Comparação relacional
`LPAREN` / `RPAREN`	`(` , `)`	Parênteses
`LBRACE` / `RBRACE`	`{` , `}`	Chaves de bloco
`SEMI`	`;`	Terminador de statement
`EOF`	*(fim do arquivo)*	Marcador de fim

Ignorados pelo lexer: espaços, tabulações, quebras de linha, comentários `// ...`

2.3 Gramática Formal (EBNF)

```
(* Programa *)
program        = statement , { statement } ;

(* Statements *)
statement      = assignment
               | if_stmt
               | while_stmt
               | print_stmt ;

assignment     = IDENT , "=" , expression , ";" ;

if_stmt        = "if" , "(" , expression , ")" , block ,
                 [ "else" , block ] ;

while_stmt     = "while" , "(" , expression , ")" , block ;

print_stmt     = "print" , "(" , expression , ")" , ";" ;

block          = "{" , { statement } , "}" ;
```

```
(* Expressões — hierarquia de precedência crescente *)
expression     = comparison ;

comparison     = additive , [ comp_op , additive ] ;

comp_op        = "==" | "!=" | "<" | ">" | "<=" | ">=" ;

additive       = multiplicative , { add_op , multiplicative } ;

add_op         = "+" | "-" ;

multiplicative = unary , { mul_op , unary } ;

mul_op         = "*" | "/" ;

unary          = "-" , unary | primary ;

primary        = INTEGER | IDENT | "(" , expression , ")" ;
```

2.4 Precedência de Operadores

Nível	Operadores	Associatividade
1 (mais baixo)	`==` , `!=` , `<` , `>` , `<=` , `>=`	Não associativo
2	`+` , `-`	Esquerda
3	`*` , `/`	Esquerda
4 (mais alto)	`-` (unário)	Direita

2.5 Exemplos de Código TinyScript

```
// Variáveis e aritmética
a = 10;
b = 3;
soma = a + b;
print(soma);

// Condicional
if (soma > 10) {
    print(1);
} else {
    print(0);
}

// Laço while
i = 0;
while (i < 5) {
```

```
    i = i + 1;
}
print(i);
```

3. 🔄 Pipeline do Compilador TinyScript

```
┌──────────────────────────────────────────────────────────────────┐
│                    COMPILADOR TINYSCRIPT                         │
│                                                                  │
│  Código-fonte (.tiny)                                            │
│        │                                                         │
│        ▼                                                         │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  FASE 1 — ANÁLISE LÉXICA                                 │    │
│  │  Módulo: lexer.py                                        │    │
│  │  Entrada: string de código-fonte                         │    │
│  │  Saída:   List[Token]                                    │    │
│  │                                                          │    │
│  │  Lexer → tokenizar() → [Token, Token, ..., EOF]         │    │
│  └──────────────────────────┬───────────────────────────────┘    │
│                             │  List[Token]                       │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  FASE 2 — ANÁLISE SINTÁTICA                              │    │
│  │  Módulo: parser.py                                       │    │
│  │  Entrada: List[Token]                                    │    │
│  │  Saída:   AST (Program node)                             │    │
│  │                                                          │    │
│  │  Parser → parse() → Program(statements=[...])            │    │
│  └──────────────────────────┬───────────────────────────────┘    │
│                             │  AST                               │
│                             ▼                                    │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  FASE 3 — GERAÇÃO DE CÓDIGO INTERMEDIÁRIO                │    │
│  │  Módulo: codegen.py                                      │    │
│  │  Entrada: AST                                            │    │
│  │  Saída:   Código TAC (string)                            │    │
│  │                                                          │    │
│  │  GeradorDeCodigo → gerar(ast) → instrucoes: str          │    │
│  └──────────────────────────┬───────────────────────────────┘    │
│                             │  Código Intermediário (TAC)        │
│                             ▼                                    │
│                    [Saída no terminal]                           │
└──────────────────────────────────────────────────────────────────┘
```

4. 📂 Estrutura dos Módulos

4.1 `ast_nodes.py` — Hierarquia de Nós

```
ASTNode (base)
├── Program          — raiz do programa
├── Block            — bloco { }
├── Assignment       — atribuição: IDENT = expr
├── IfStatement      — if (cond) block [else block]
├── WhileLoop        — while (cond) block
├── PrintStatement   — print(expr)
├── BinaryOp         — expr op expr
├── UnaryOp          — op expr
├── IntegerLiteral   — 42
└── Variable         — nome_da_variavel
```

4.2 `lexer.py` — Classes principais

```
TokenType (Enum)
    ├── INTEGER, IDENT
    ├── IF, ELSE, WHILE, PRINT
    ├── PLUS, MINUS, STAR, SLASH
    ├── ASSIGN, EQ, NEQ, LT, GT, LE, GE
    ├── LPAREN, RPAREN, LBRACE, RBRACE, SEMI
    └── EOF

Token
    ├── tipo: TokenType
    ├── valor: str | int
    ├── linha: int
    └── coluna: int

Lexer(codigo_fonte: str)
    └── tokenizar() → List[Token]
```

4.3 `parser.py` — Parser Recursivo Descendente

```
ErroSintatico (Exception)

Parser(tokens: List[Token])
    ├── parse()                  → Program
    ├── _parse_program()         → Program
    ├── _parse_statement()       → ASTNode
    ├── _parse_assignment()      → Assignment
    ├── _parse_if()              → IfStatement
    ├── _parse_while()           → WhileLoop
```

```
    ├── _parse_print()            → PrintStatement
    ├── _parse_block()            → Block
    ├── _parse_expression()       → ASTNode
    ├── _parse_comparison()       → ASTNode
    ├── _parse_additive()         → ASTNode
    ├── _parse_multiplicative()   → ASTNode
    ├── _parse_unary()            → ASTNode
    └── _parse_primary()          → ASTNode
```

4.4 `codegen.py` — Gerador de Código TAC

```
GeradorDeCodigo
    ├── gerar(ast) → str          ← interface pública
    ├── _visitar(no)              ← dispatch por tipo
    ├── _novo_temp() → "_t0"      ← variáveis temporárias
    ├── _novo_label() → "L0"      ← rótulos de desvio
    │
    ├── _visitar_Program
    ├── _visitar_Block
    ├── _visitar_Assignment       → emite: "x = _t0"
    ├── _visitar_IfStatement      → emite: IF NOT / GOTO / labels
    ├── _visitar_WhileLoop        → emite: labels / IF NOT / GOTO
    ├── _visitar_PrintStatement   → emite: "PRINT _t1"
    ├── _visitar_BinaryOp         → emite: "_t2 = _t0 + _t1"
    ├── _visitar_UnaryOp          → emite: "_t3 = - _t2"
    ├── _visitar_IntegerLiteral   → emite: "_t0 = 42"
    └── _visitar_Variable         → retorna nome (sem instrução)
```

5. 🌳 Exemplo: Código → Tokens → AST → TAC

Código de entrada

```
x = 5 + 3;
if (x > 6) {
    print(x);
}
```

Tokens produzidos (Fase 1)

```
Token(IDENT,   'x',  L1:C1)
Token(ASSIGN,  '=',  L1:C3)
Token(INTEGER, 5,    L1:C5)
Token(PLUS,    '+',  L1:C7)
```

```
Token(INTEGER, 3,    L1:C9)
Token(SEMI,    ';',  L1:C10)
Token(IF,      'if', L2:C1)
Token(LPAREN,  '(',  L2:C4)
Token(IDENT,   'x',  L2:C5)
Token(GT,      '>',  L2:C7)
Token(INTEGER, 6,    L2:C9)
Token(RPAREN,  ')',  L2:C10)
Token(LBRACE,  '{',  L2:C12)
Token(PRINT,   'print', L3:C5)
...
```

AST produzida (Fase 2)

```
Program
├── Assignment [x]
│   └── BinaryOp [+]
│       ├── IntegerLiteral [5]
│       └── IntegerLiteral [3]
└── If
    ├── condição:
    │   └── BinaryOp [>]
    │       ├── Variable [x]
    │       └── IntegerLiteral [6]
    └── then:
        └── Block
            └── Print
                └── Variable [x]
```

Código intermediário TAC (Fase 3)

```
_t0 = 5
_t1 = 3
_t2 = _t0 + _t1
x = _t2
_t3 = 6
_t4 = x > _t3
IF NOT _t4 GOTO L0
PRINT x
GOTO L1
L0:
L1:
```

6. 🚦 Tratamento de Erros

Fase	Tipo de Erro	Exemplo	Mensagem
Léxica	`ErroLexico`	`@x = 1;`	`Erro léxico na linha 1, coluna 1: caractere inesperado '@'`
Sintática	`ErroSintatico`	`x = ;`	`Erro sintático na linha 1. coluna 5: expressão inválida: token inesperado SEMI`
Sintática	`ErroSintatico`	`if x > 1 { }`	`Erro sintático na linha 1, coluna 4: esperado LPAREN`

7. 🔮 Extensões Futuras

Extensão	Descrição
Tipos `float` e `string`	Ampliar o sistema de tipos
Funções	`func nome(params) { ... } / return`
Tabela de símbolos	Verificação semântica (variáveis não declaradas)
Otimização TAC	Eliminação de temporários redundantes
Geração de Python	Traduzir TAC para código Python executável
REPL interativo	Loop read-eval-print para TinyScript

📚 Referências

1. **AHO, A. V. et al.** Compiladores: princípios, técnicas e ferramentas. 2. ed. Pearson, 2008.
2. **MENDEZ, P.; BLAUTH, P.** Linguagens formais e autômatos. 6. ed. Bookman, 2010.
3. **SANTOS, P. R.; LANGLOIS, T.** Compiladores: da teoria à prática. LTC, 2019.
4. **GRUNE, D. et al.** Modern Compiler Design. 2. ed. Springer, 2012.